我们分析 Lua 虚拟机肯定不是完全把 86 条指令集全部分析出来，那得一个多月了，选择几个类别的，比较有特点的东西来写成文章进行分析，一下出自 AI 老爷，因为我懒得自己想了

Level 0：纯寄存器搬运（零抽象）

✅ OP_MOVE

难度：★☆☆☆☆

解决的问题

- 寄存器 = 栈槽
- base + offset 的真实含义

你能学到

- RA/RB 是如何变成内存地址的
- `base` 是否被缓存到真实寄存器
- `setobj` 是否被内联

GDB 必要性：⭐⭐⭐⭐⭐（必上）

停在哪

- 看清 `ra/rb` 的地址计算即可
- 不要深入 GC

Level 1：立即数 / 常量加载

✅ OP_LOADI / OP_LOADK

难度：★☆☆☆☆

解决的问题

- 常量是“值”还是“引用”
- K 表访问方式

你能学到

- `Proto->k` 的内存布局
- 常量访问是否 inline
- `LOADKX + EXTRAARG` 的设计动机

GDB 必要性：⭐⭐⭐⭐

停在哪

- 看清 K 表寻址
- 不追字符串 intern

Level 2：简单算术（无对象）

✅ OP_ADDI / OP_ADD

难度：★★☆☆☆

解决的问题

- 数值系统分支
- int / float 快路径

你能学到

- Lua 的“先试整数”策略
- fast path vs slow path
- 何时掉进 metamethod

GDB 必要性： ⭐⭐⭐

停在哪

- 不深挖 metamethod

Level 3：比较与条件跳转

✅ OP_EQ / OP_TEST / OP_JMP

难度： ★★☆☆☆

解决的问题

- 控制流
- pc 的真实身份

你能学到

- pc++ 是普通变量，不是寄存器
- branch 在解释器中的代价
- 为什么 jump table 很重要

GDB 必要性： ⭐⭐⭐⭐

停在哪

- 不做分支预测微优化

Level 4：表读取（快慢路径）

✅ OP_GETI / OP_GETFIELD / OP_GETTABLE

难度： ★★★☆☆

解决的问题

- table 的双结构
- inline cache

你能学到

- array part / hash part
- short string 的特殊优化
- miss 以后怎么 fallback

GDB 必要性： ⭐⭐⭐⭐⭐

停在哪

- 不追 hash 冲突细节

Level 5：表写入 + GC 写屏障

✅ OP_SETI / OP_SETTABLE

难度： ★★★☆☆

解决的问题

- GC 正确性

你能学到

- old → young 写屏障
- barrier 为何不能省
- 为什么写比读复杂

GDB 必要性： ⭐⭐⭐⭐

停在哪

- 不深入 GC sweep

Level 6：一元操作 / 元方法入口

⚠️ OP_UNM / OP_LEN / OP_NOT

难度： ★★★☆☆

解决的问题

- 类型驱动执行

你能学到

- `tt_` 的核心地位
- metamethod 触发点

GDB 必要性： ⭐⭐

停在哪

- 不深挖 __len 等元表

Level 7：函数调用（结构跃迁）

🔥 OP_CALL

难度：★★★★☆

解决的问题

- VM 栈帧切换

你能学到

- CallInfo 链
- 参数 / 返回值布局
- tailcall 的本质

GDB 必要性：⭐⭐⭐⭐⭐（重点）

停在哪

- 不一次性搞懂 coroutine

Level 8：函数返回（控制权回收）

🔥 OP_RETURN

难度：★★★★☆

解决的问题

- VM 恢复现场

你能学到

- savedpc 恢复
- 栈回缩
- return0 / return1 的优化意义

GDB 必要性： ⭐⭐⭐⭐

停在哪

- 不追异常路径

Level 9：Upvalue / 闭包（环境捕获）

⚠️ OP_GETUPVAL / OP_SETUPVAL / OP_CLOSURE

难度： ★★★★★

解决的问题

- 变量逃逸

你能学到

- open / closed upvalue
- 栈 → 堆迁移

GDB 必要性： ⭐⭐⭐⭐⭐

停在哪

- 不重写 closure 系统

Level 10：控制结构糖衣（最后再看）

❌ FOR / TFOR / VARARG / SETLIST

难度：★★★★★

说明

- 它们不是新结构
- 是前面所有东西的组合

👉 **最后看，甚至可以不看**

总结

分析 Lua 虚拟机并不等同于穷举其全部 86 条字节码指令。那样得到的只是一份“操作码说明书”，而不是对 VM 本质的理解。本系列刻意按**结构复杂度与抽象层级**对指令进行分层选取：从纯寄存器搬运，到常量系统、算术快慢路径、控制流、表结构、GC 写屏障，再到函数调用与闭包捕获。每一层都对应 Lua VM 中一个**不可回避的设计决策**，而不是语法糖。

关注的重点始终不是“Lua 语义上它做了什么”，而是：
这一条字节码在解释器里如何落到真实内存、真实跳转、真实栈帧变化上。因此分析会在“足以看清执行模型”的位置停下，刻意回避 GC、hash 冲突、元方法完整语义等深水区。目标只有一个——建立对 Lua VM 执行引擎的**二进制级直觉**，而不是写一份更长的官方文档。

当你真正理解了 OP_CALL、OP_RETURN、Upvalue 这些“结构跃迁点”，剩下的指令就只是在这些骨架上堆叠出来的组合而已